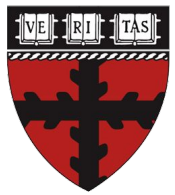
Lecture 12: App Design

AC215/E115

Pavlos Protopapas
Shivas Jayaram



Logistics/Reminders

✅ Please make sure to fill group info spreadsheet. Almost everyone has partners 🎉

📌 MS1 is due tonight 9/25 9PM

📚 HW1 has been released - Office Hours schedule will be posted on Ed

☁️ If your group plans to use AWS for projects → fill out this form by tonight (9/25, 10 PM)

🐌 Still waiting on GCP credits.

Outline

1. Recap
2. Motivation
3. App Design
4. Screenflow & Wireframes
5. Solution Architecture
6. Technical Architecture
7. Setup & Code Organization

Outline

1. **Recap**
2. Motivation
3. App Design
4. Screenflow & Wireframes
5. Solution Architecture
6. Technical Architecture
7. Setup & Code Organization

Recap: 🧀 Cheese App

- We want to build an app to identify a cheese by simply taking a photo of it
- Dive deeper into the world of cheese with our interactive chatbot
- How do we build the app?

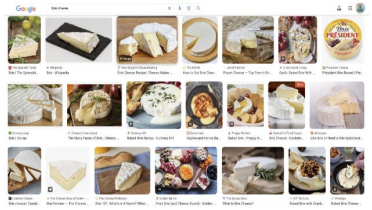


Recap: How do we build an App?

- Collaborate with team to design and develop.
- Build a robust ml pipeline for data and models.
- Expose python functions as backend APIs.
- Build a frontend using HTML & javascript.
- Deploy app to a cloud provider.
- http://formaggio.me/ [Go live]

Recap: How do we build an App?

Development



Python Script

Python: pipenv
Chromium: Mac install, Windows install
OS: Mac, Windows



Colab

Python: Colab provided env
OS: Linux

Rest API

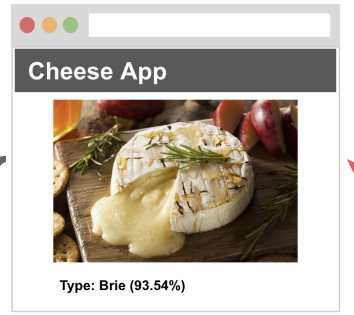
Best Model



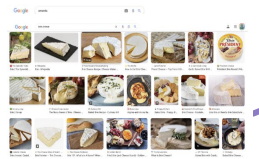
IDE / Code Editor

Python: pipenv
OS: Mac, Windows

Deployment



Model Training



Python Script

Python: pipenv
OS: Linux

Server

Multiple developers, Using Mac and Windows OS

Outline

1. Recap
2. **Motivation**
3. App Design
4. Screenflow & Wireframes
5. Solution Architecture
6. Technical Architecture
7. Setup & Code Organization

Before you build your App

- First we build an **ML Pipeline**
- We want to build an app that uses the **ML Components**
- Expose model and python functions as **APIs**
- Identify **user needs** that can be fulfilled by APIs
- Design **user interface** needs

How do we do this?

Review: Problem Definition

Imagine being able to identify a cheese by simply taking a photo of it. Our app uses AI-powered visual recognition technology to help you identify the cheese you're looking at, and then provides you with a wealth of information about it.

Take a photo of the cheese, and our app will identify it for you. Then, dive deeper into the world of cheese with our interactive chatbot. Ask questions about the cheese's origin, production process, nutritional information, and history. Get even more information using customized newsletters and podcasts.

Review: Proposed Solution

Key Features:

- Visual cheese identification using AI-powered technology
- Interactive chatbot for asking questions about cheese
- In-depth information on cheese origin, production process, nutritional information, and history
- Expert advice on pairing cheese with wines, crackers, and other accompaniments
- Perfect for cheese enthusiasts, party planners, and anyone looking to explore the world of cheese
- Customized newsletters and podcasts by user interest.

Review: Project Scope



Proof Of Concept (POC)

- Scrap cheese images and documents (books etc)
- Verify images and pdfs
- Experiment on some baseline models
- Verify new unseen cheeses are predicted by the model(s)
- Verify ideas using an LLM

Prototype

- **Create a mockup of screens to see how the app could look like**
- **Deploy one model to Fast API to service model predictions as an API**

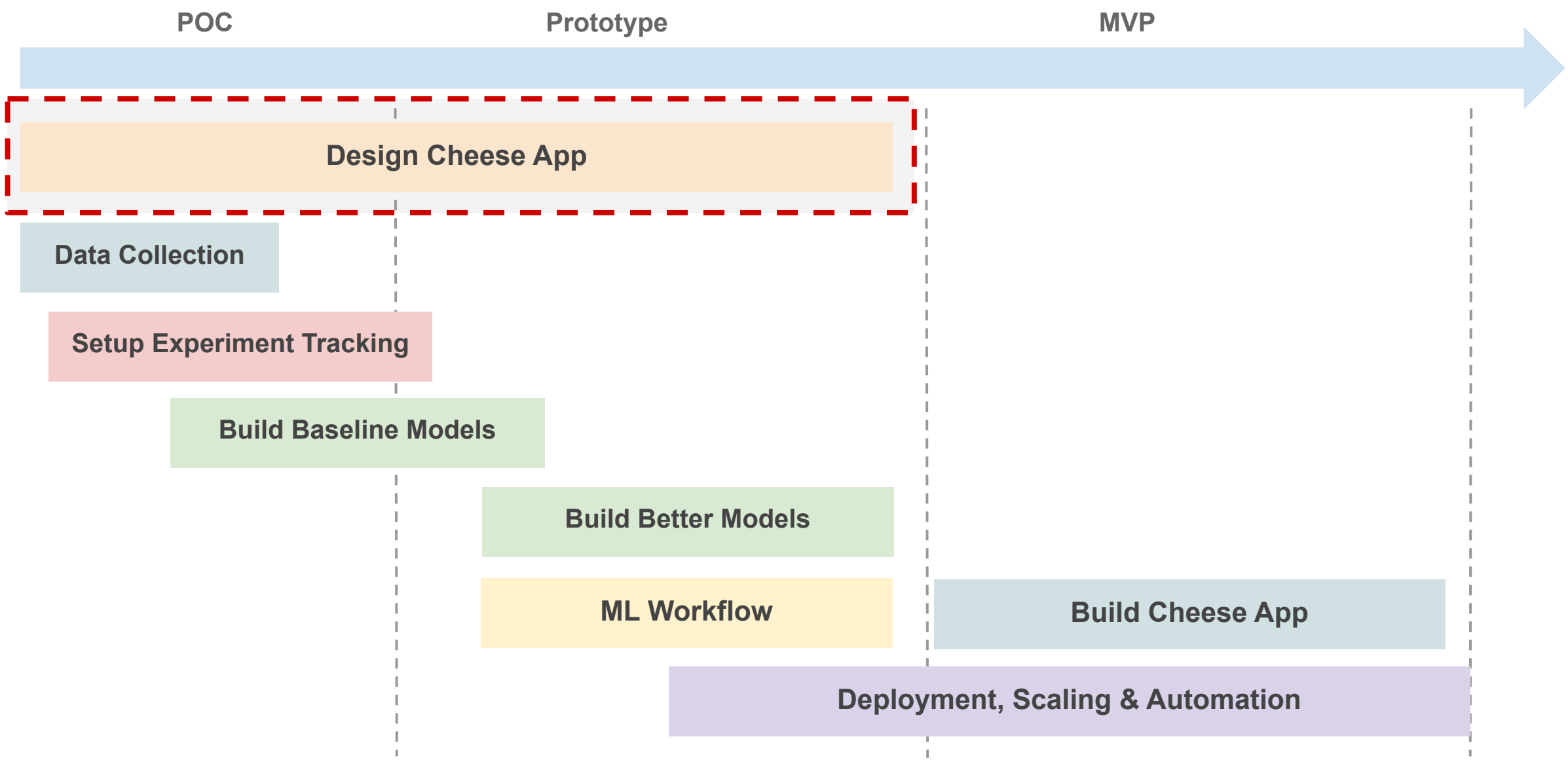
Minimum Viable Product (MVP)

- **Create App to identify Cheeses and respond appropriately to a series of prompts**
- **API Server for uploading images and predicting using best model**
- **API Server for serving the language models**
- **Newsletters**
- **Podcasts**

Using Streamlit

Using Web Frameworks

Review: Cheese App Status



Outline

1. Recap
2. Motivation
3. **App Design**
4. Screenflow & Wireframes
5. Solution Architecture
6. Technical Architecture
7. Setup & Code Organization

App Design

- In a traditional software app you have code and data.
- In an **AI App**, in addition you have models to perform tasks
- For apps that use **LLMs** we have prompts
- We will follow a structured approach to design and develop an AI App
- The design will consist of the following components:
  - Screenflow & Wireframes
  - Solution Architecture
  - Technical Architecture

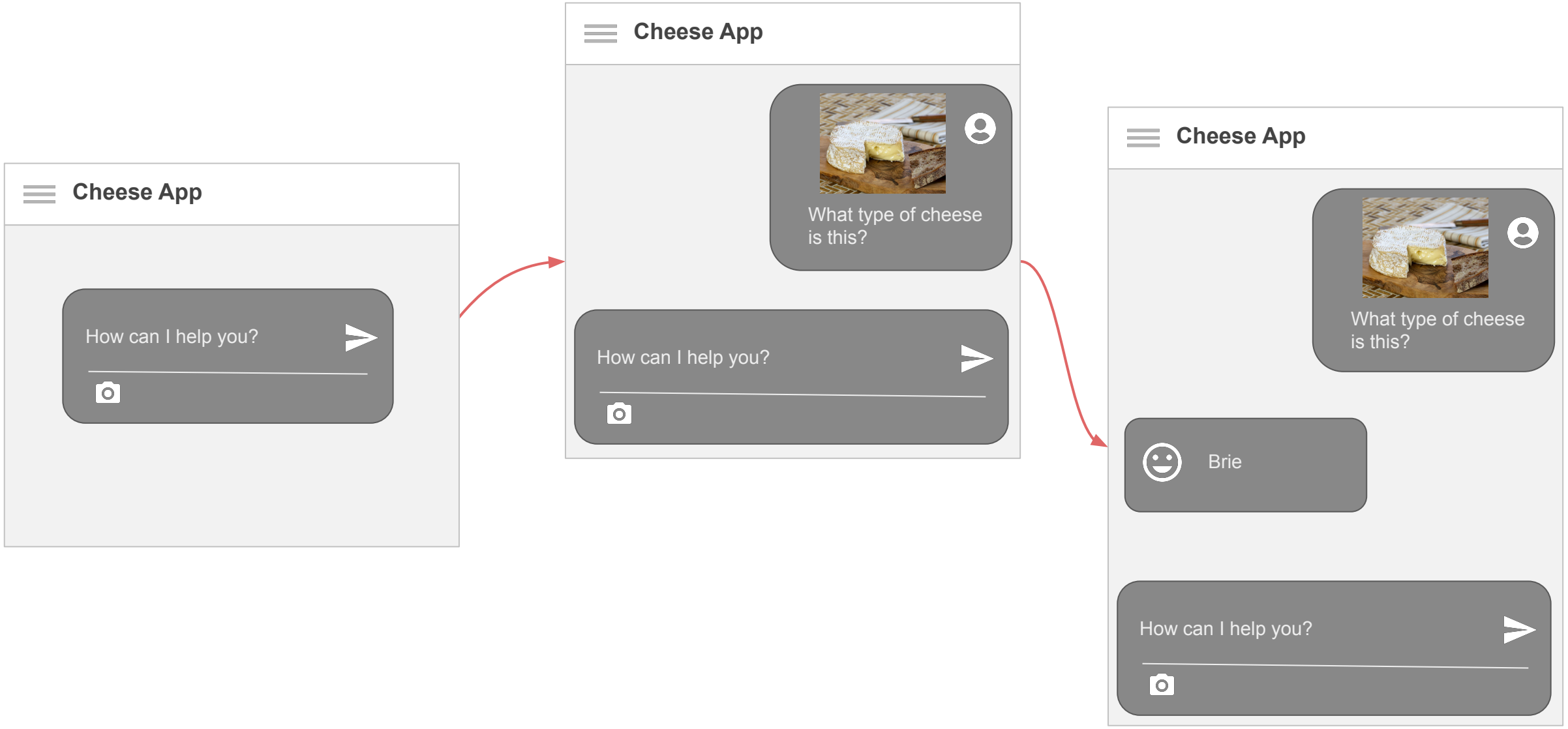
Outline

1. Recap
2. Motivation
3. App Design
4. **Screenflow & Wireframes**
5. Solution Architecture
6. Technical Architecture
7. Setup & Code Organization

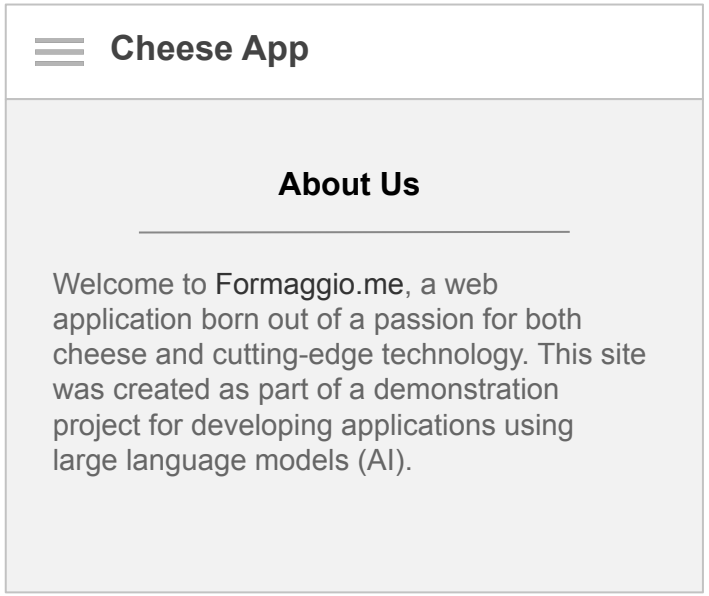
Screenflow & Wireframes

Start with brainstorming ideas on whiteboard/paper

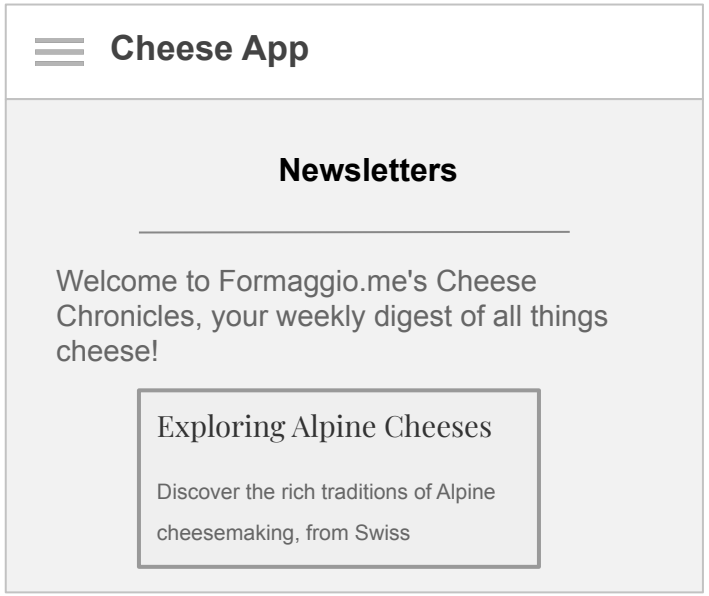
Screenflow & Wireframes



Screenflow & Wireframes







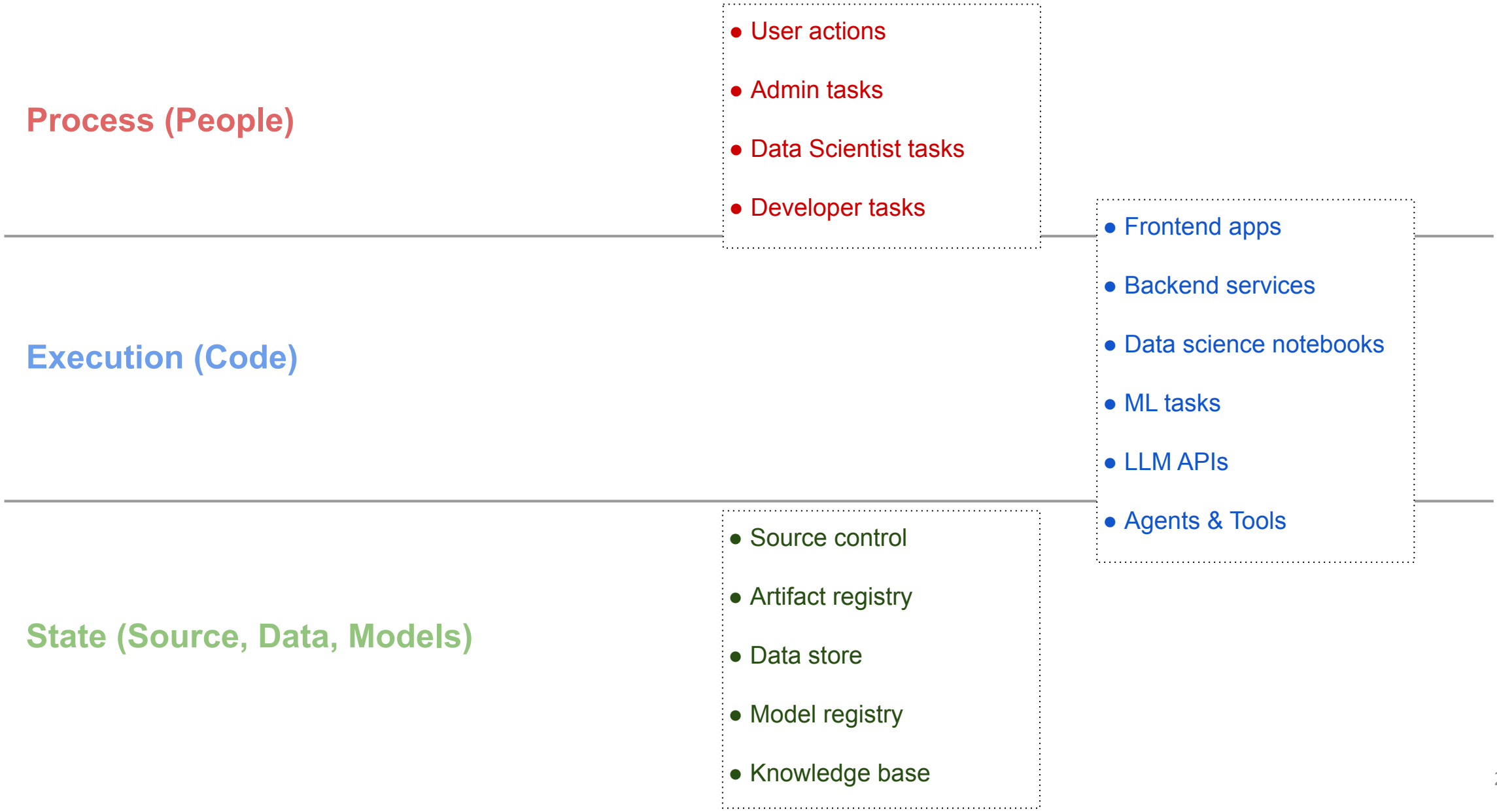
Outline

1. Recap
2. Motivation
3. App Design
4. Screenflow & Wireframes
5. **Solution Architecture**
6. Technical Architecture
7. Setup & Code Organization

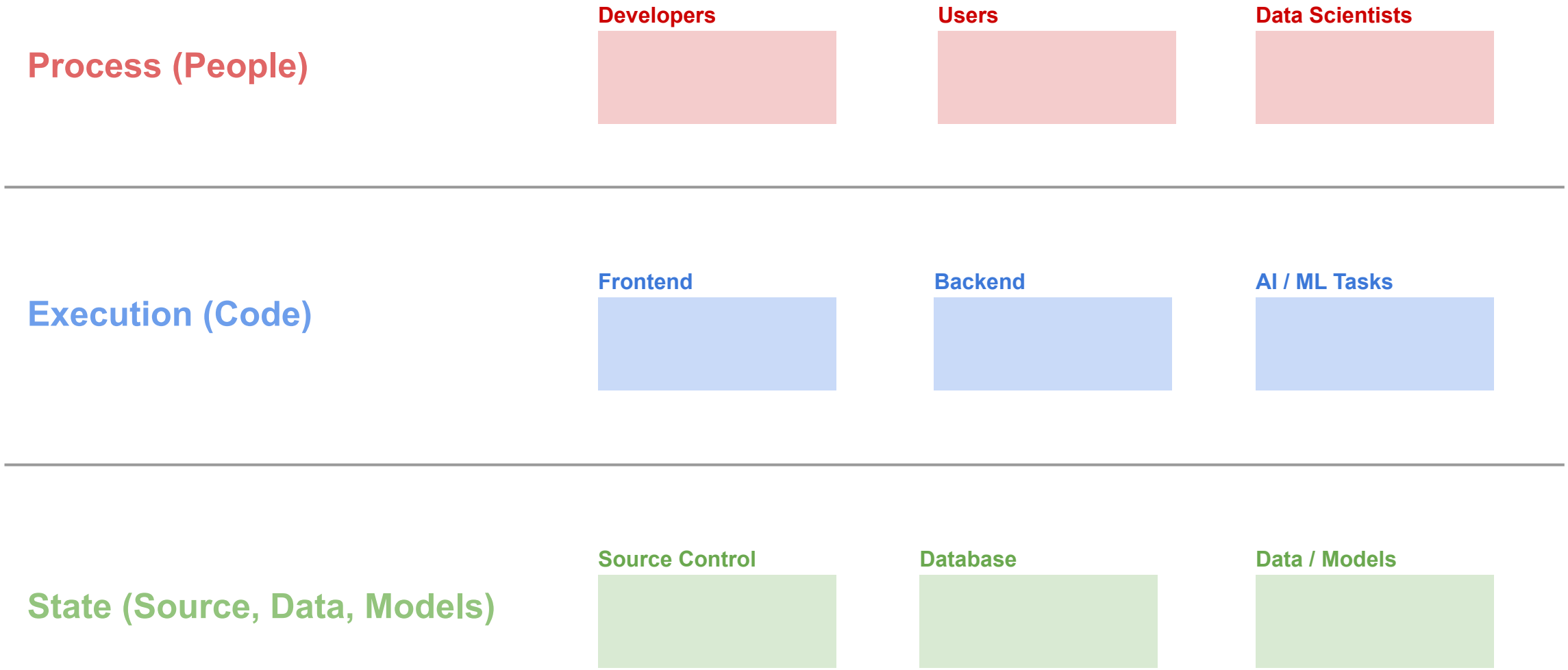
Solution Architecture

- Helps to identify the building **blocks** in an App
- Start by asking how will your **App** address the **Problem Statement**
- Identifying the following:
  - The **Process** being performed by the user
  - The code **Execution** blocks required to fulfil the **Process**
  - The **State** required during the life cycle of the App

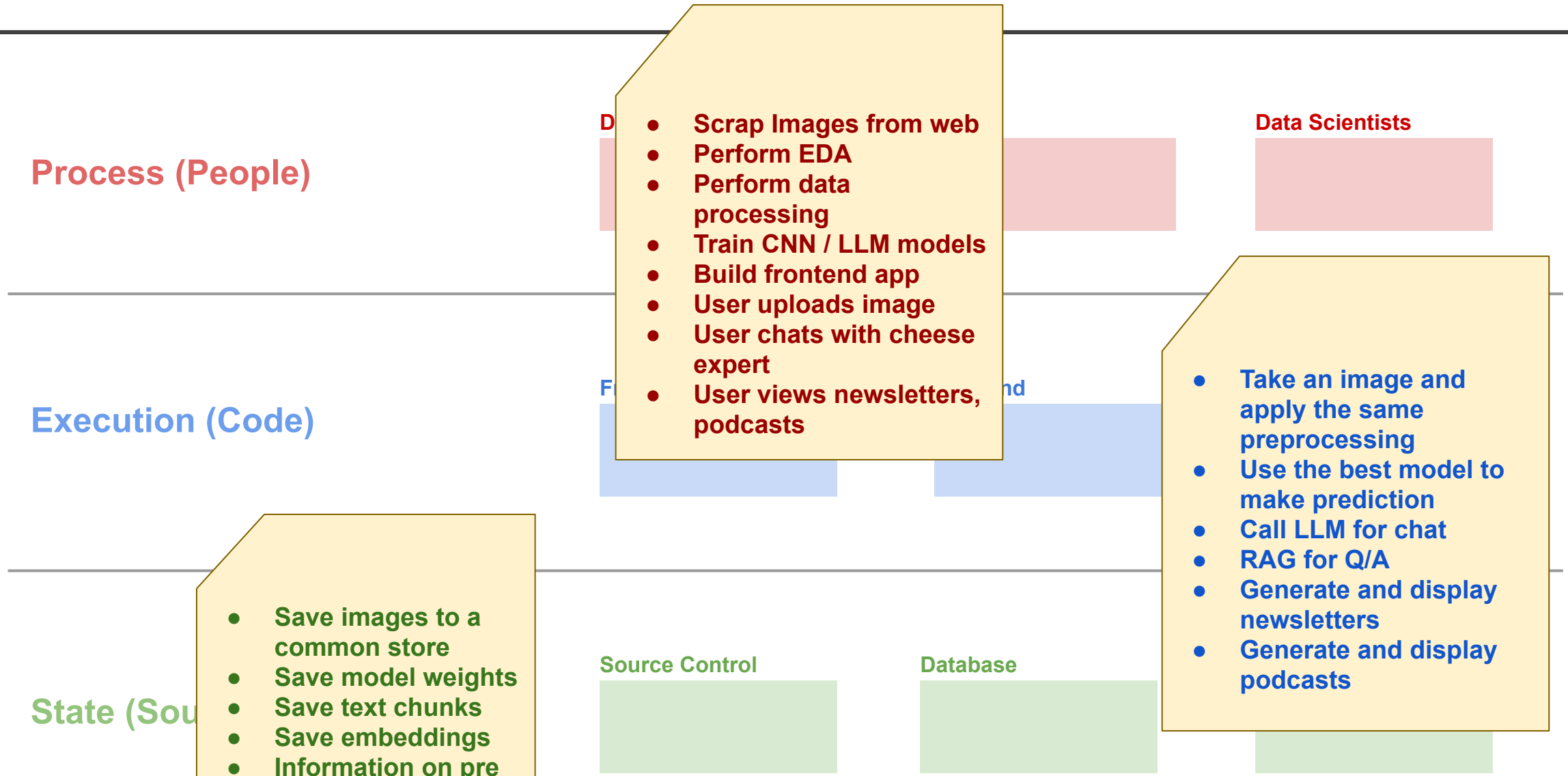
Solution Architecture



Solution Architecture AI App



Solution Architecture



Solution Architecture

Process

Execution

State

Solution Architecture

Process



Develop App

AI/ML Tasks

Upload picture, chat, etc.

Execution

State

Solution Architecture

Process



Develop App

AI/ML Tasks

Upload picture, view predictions

Execution

State



Source Control



Artifact Registry



Data Store

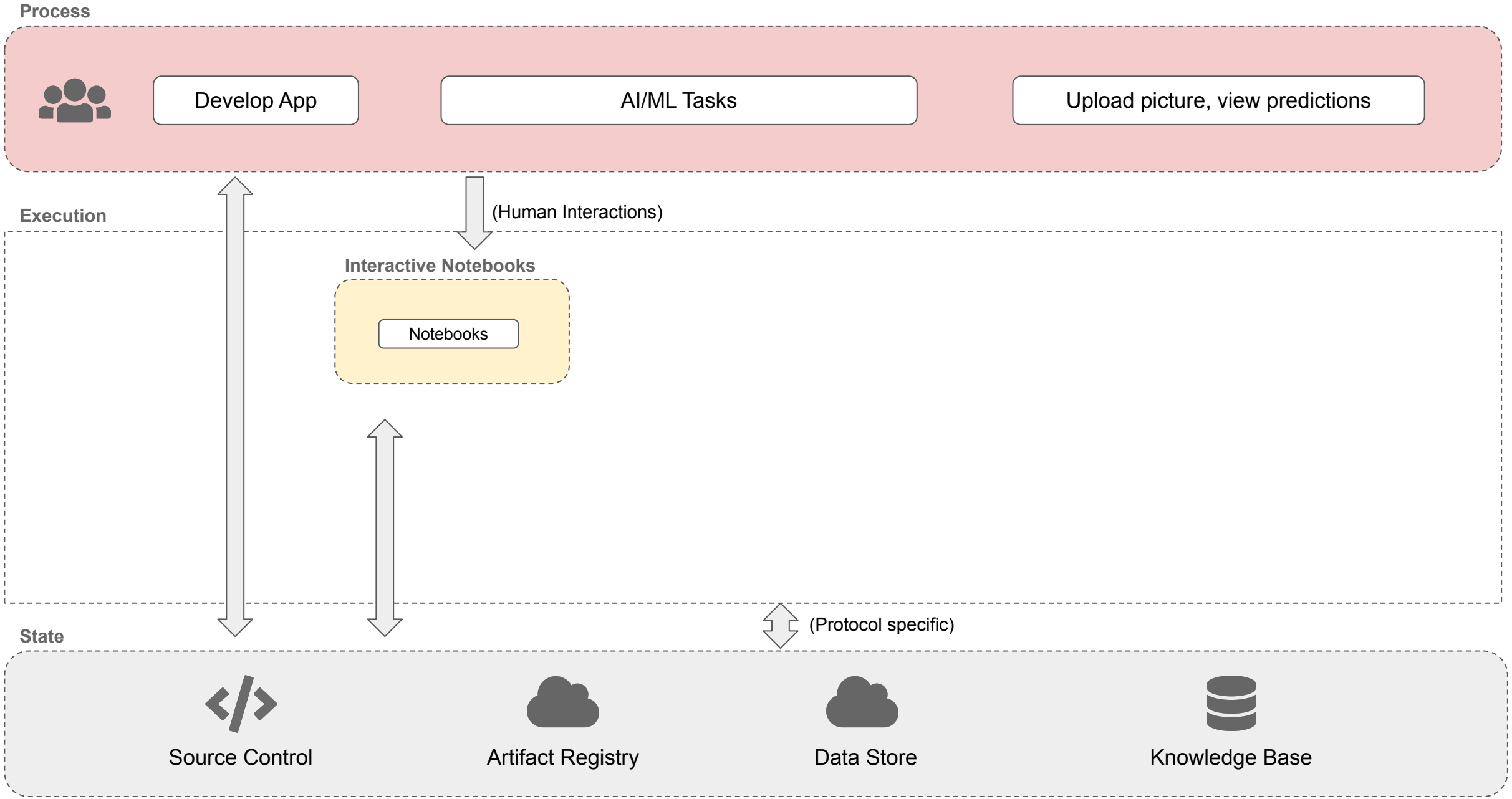


Knowledge Base

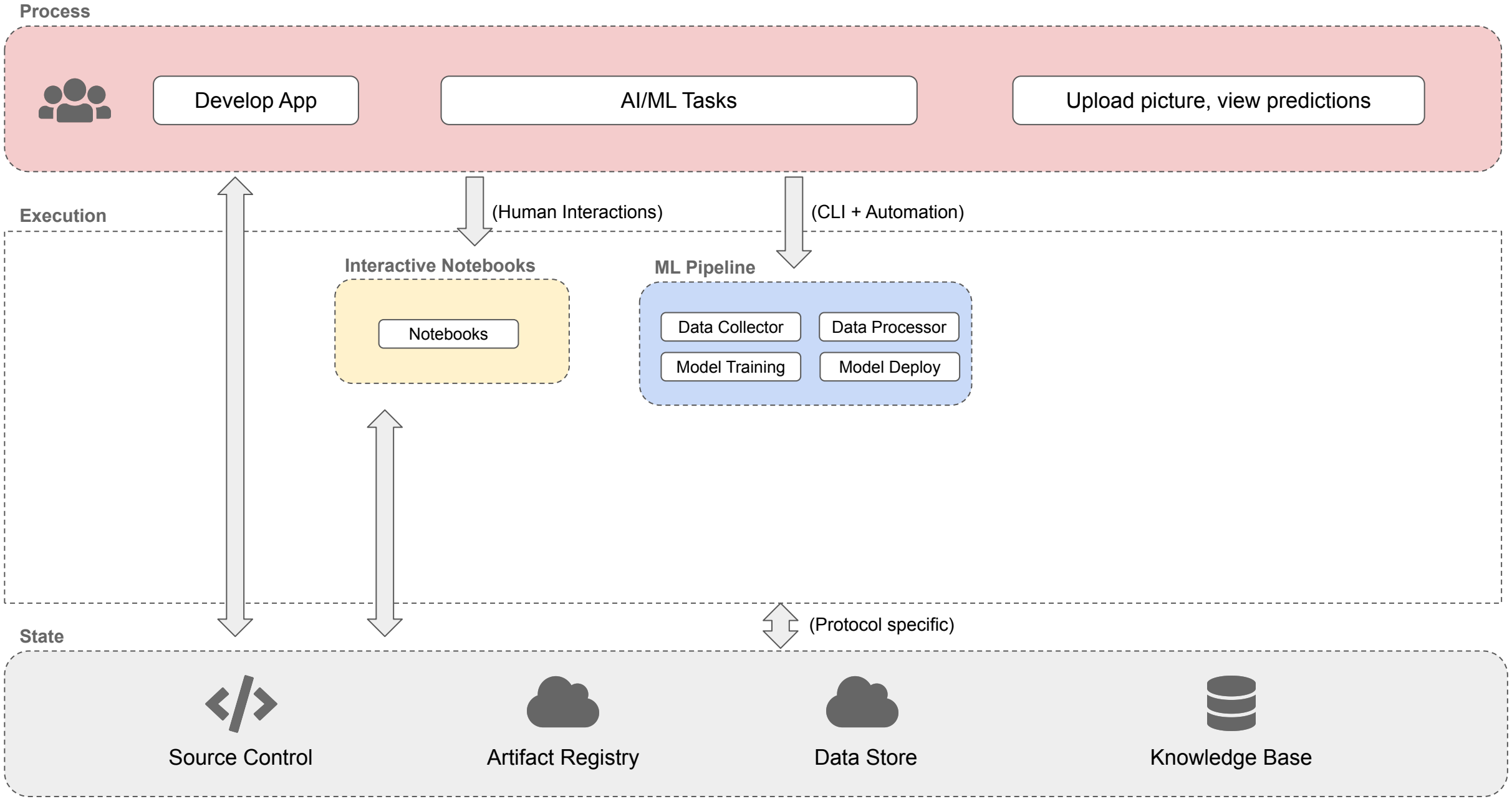
Solution Architecture



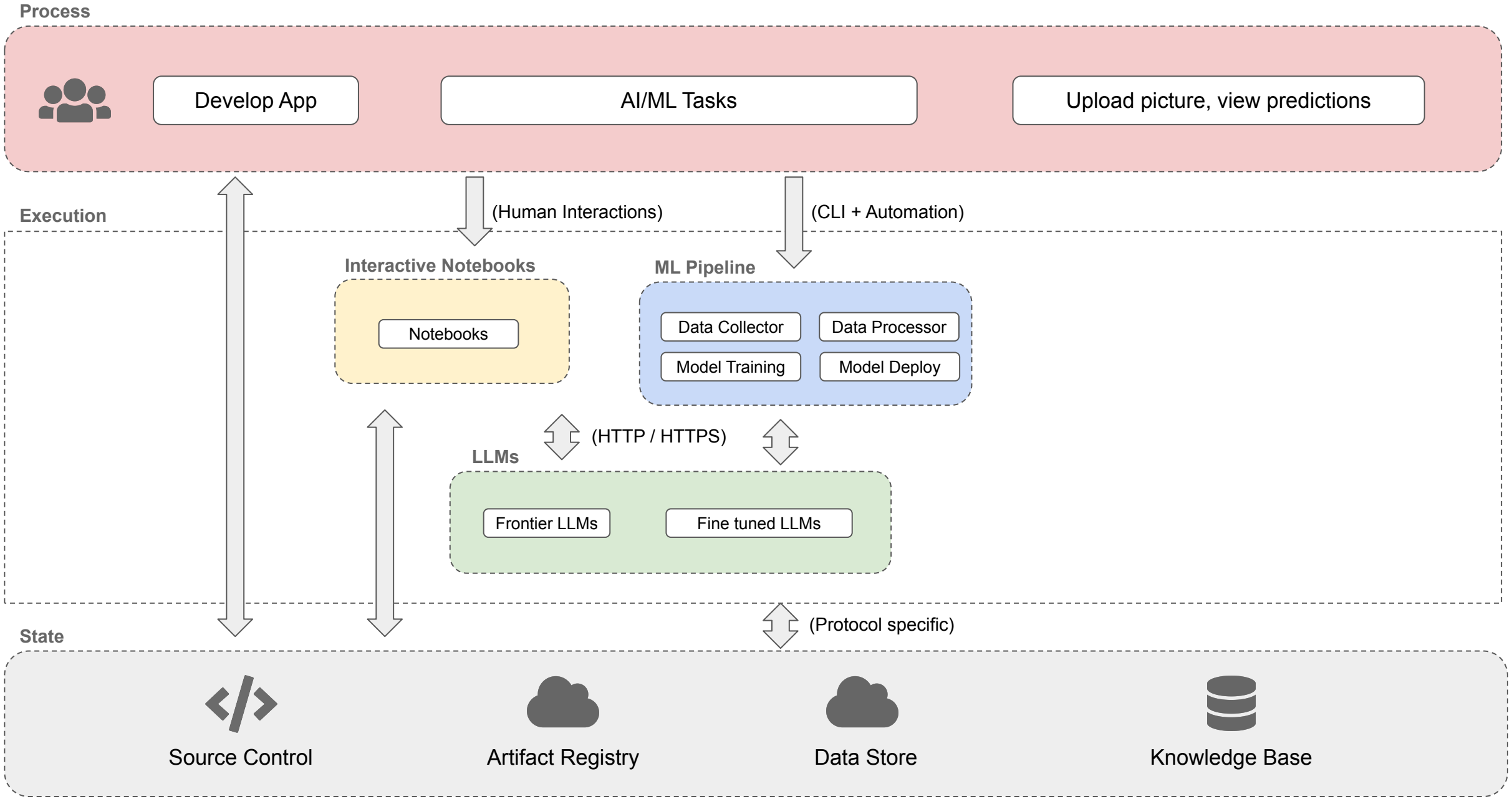
Solution Architecture



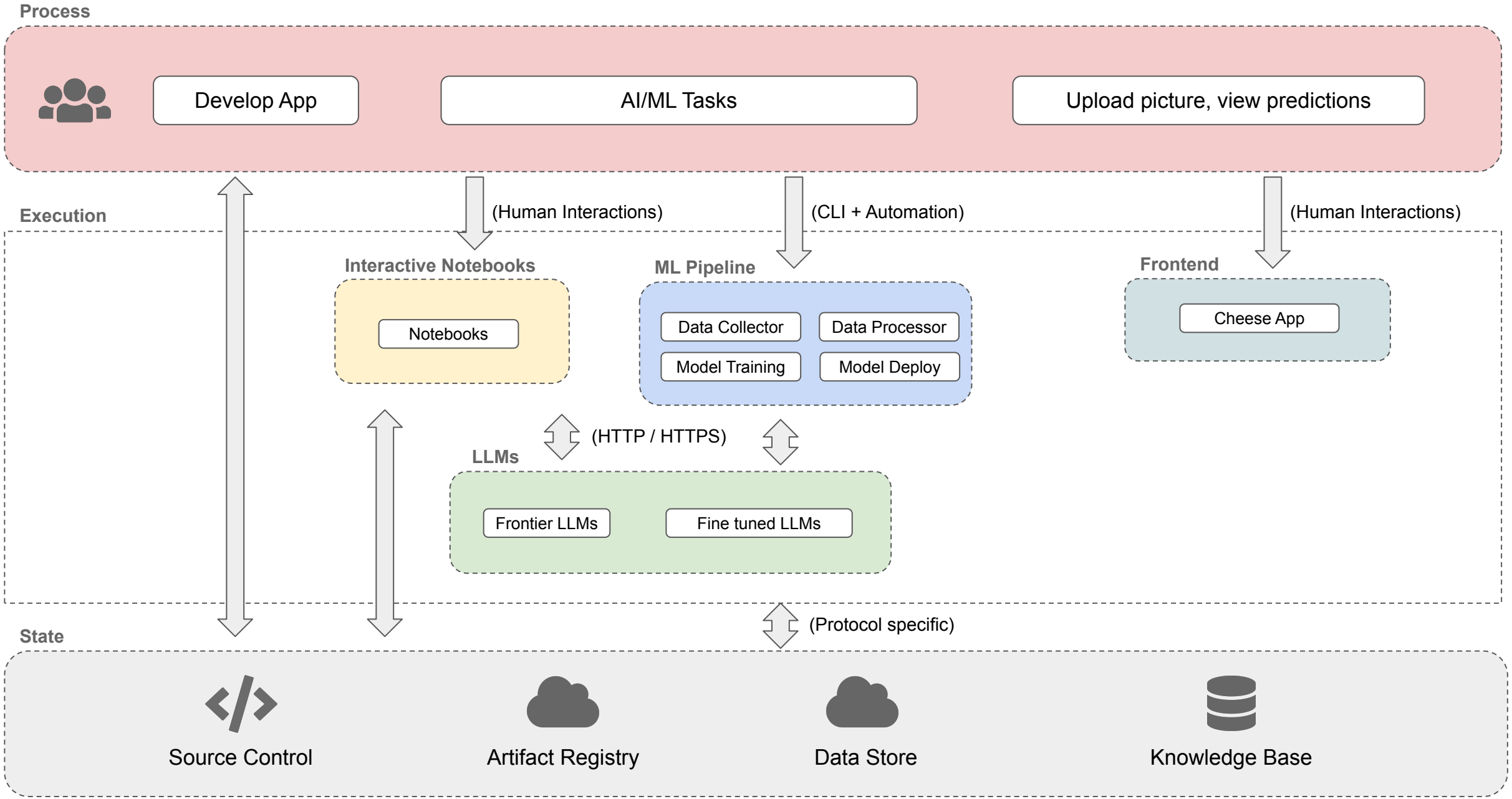
Solution Architecture



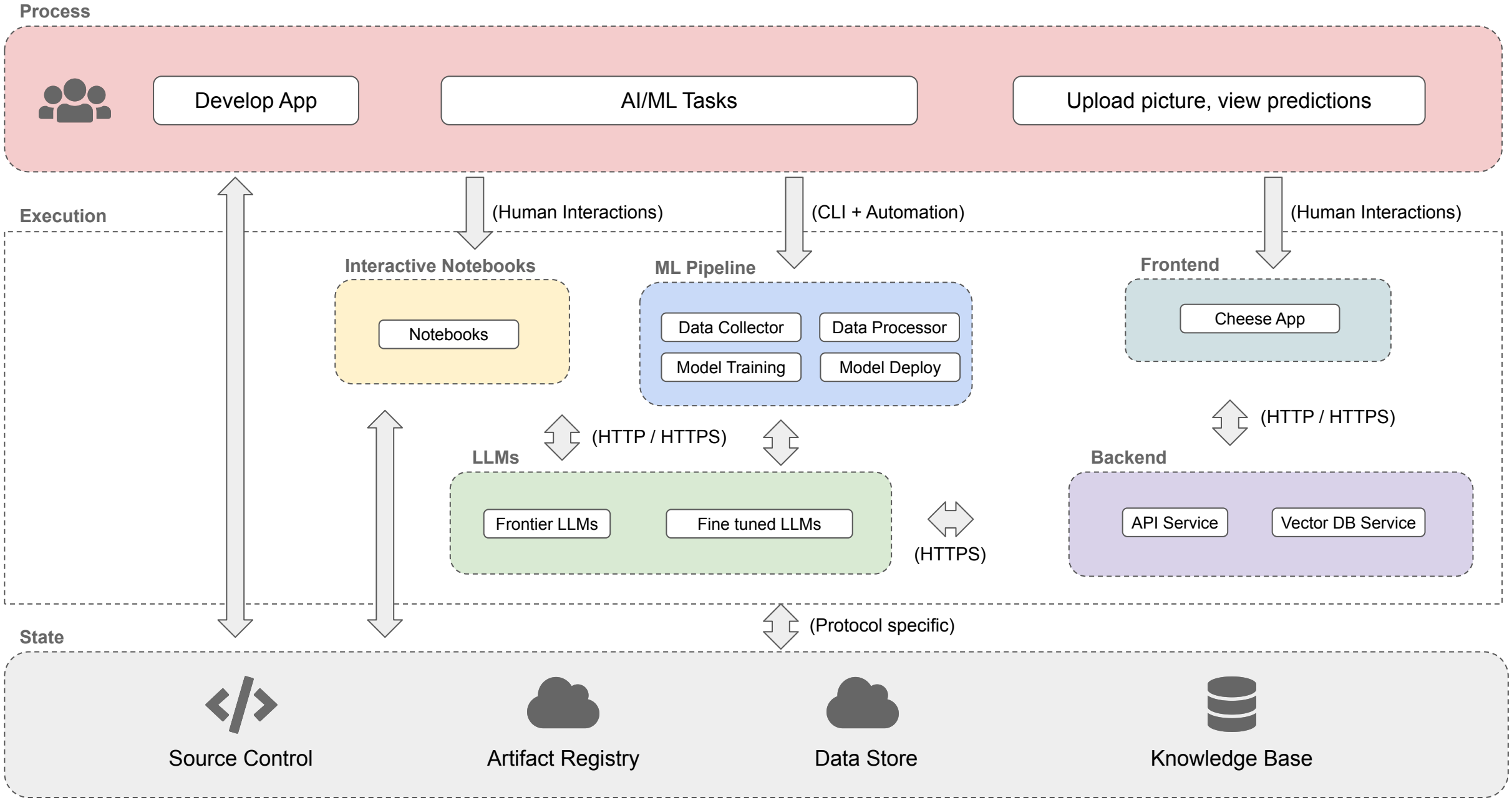
Solution Architecture



Solution Architecture



Solution Architecture



Solution Architecture Summary

- **Process**
  - Data Scientists perform ML Tasks
  - Developers build App
  - Users can upload pictures and have a chat conversation
- **Interactive Notebooks**
  - Web based hosted notebook solution to experiment ML task
- **ML Pipeline**
  - Containerized ML components
  - Helps to automate and run ML tasks
- **LLMs**
  - Frontier LLM APIs
  - Finetuned Frontier LLM APIs
- **Frontend**
  - User friendly single page app with capabilities to upload and chat with backend
- **Backend**
  - API server to expose python functions to frontend
- **State**
  - Source control to store/version code
  - Artifact registry for docker images
  - Image store for data
  - Models and model artifacts store

Outline

1. Recap
2. Motivation
3. App Design
4. Screenflow & Wireframes
5. Solution Architecture
6. **Technical Architecture**
7. Setup & Code Organization

Technical Architecture

- Helps design and develop an **AI App**
- High level view from **development** to **deployment**
- Illustrates **interactions** between components/**containers**
- **Blueprint** of the system
  - Helps team members understand the big picture
  - Helps onboarding new team members

Building a Technical Architecture



Building a Technical Architecture



Building a Technical Architecture



Developers:

- Use IDE (VSCode), CLI to build app components
- All development is containerized

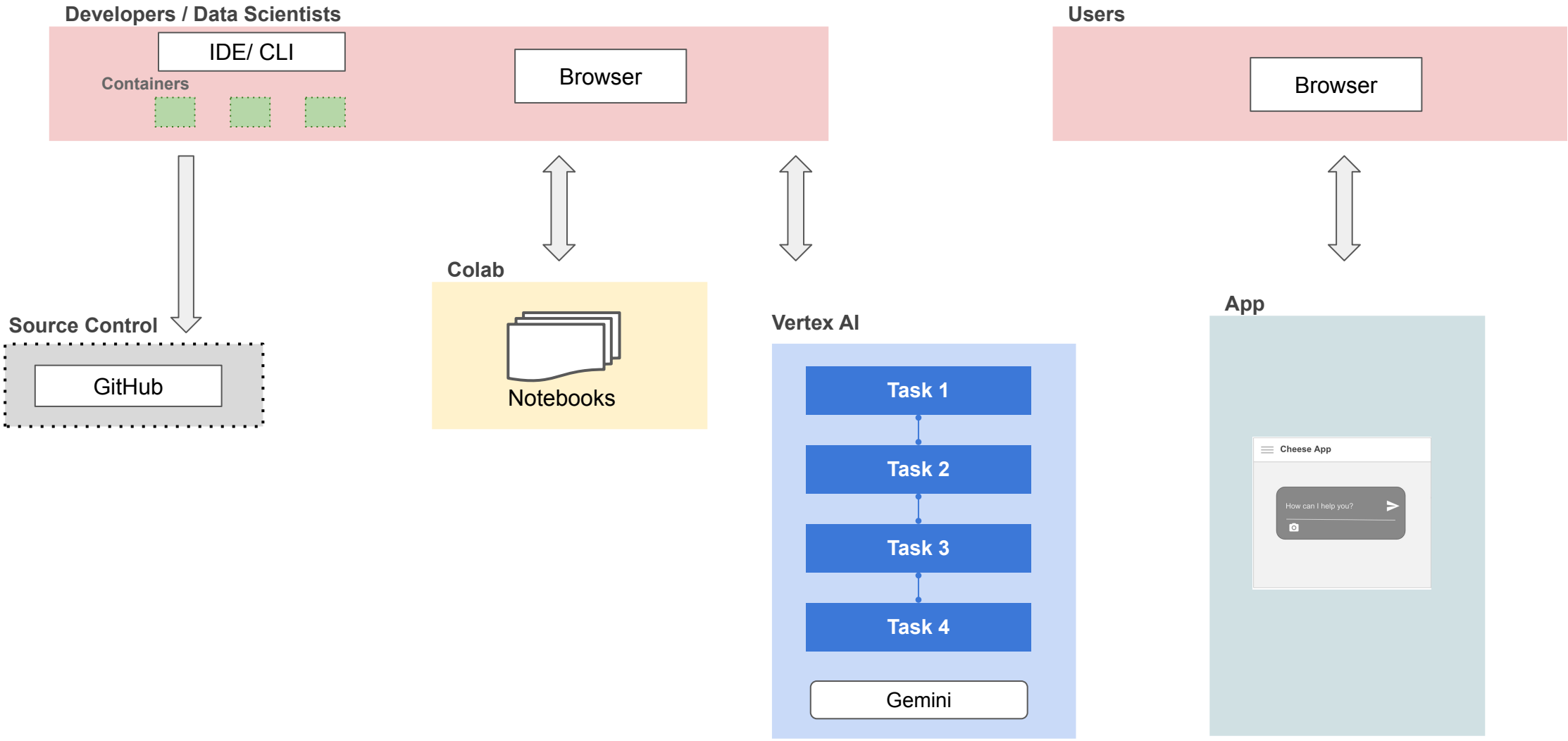
Data Scientists:

- Use Colab/JupyterHub
- EDA on notebooks
- Data & Model experimentation on notebooks
- Use IDE (VSCode), CLI to build ML Tasks
- All development is containerized
- Access LLMs

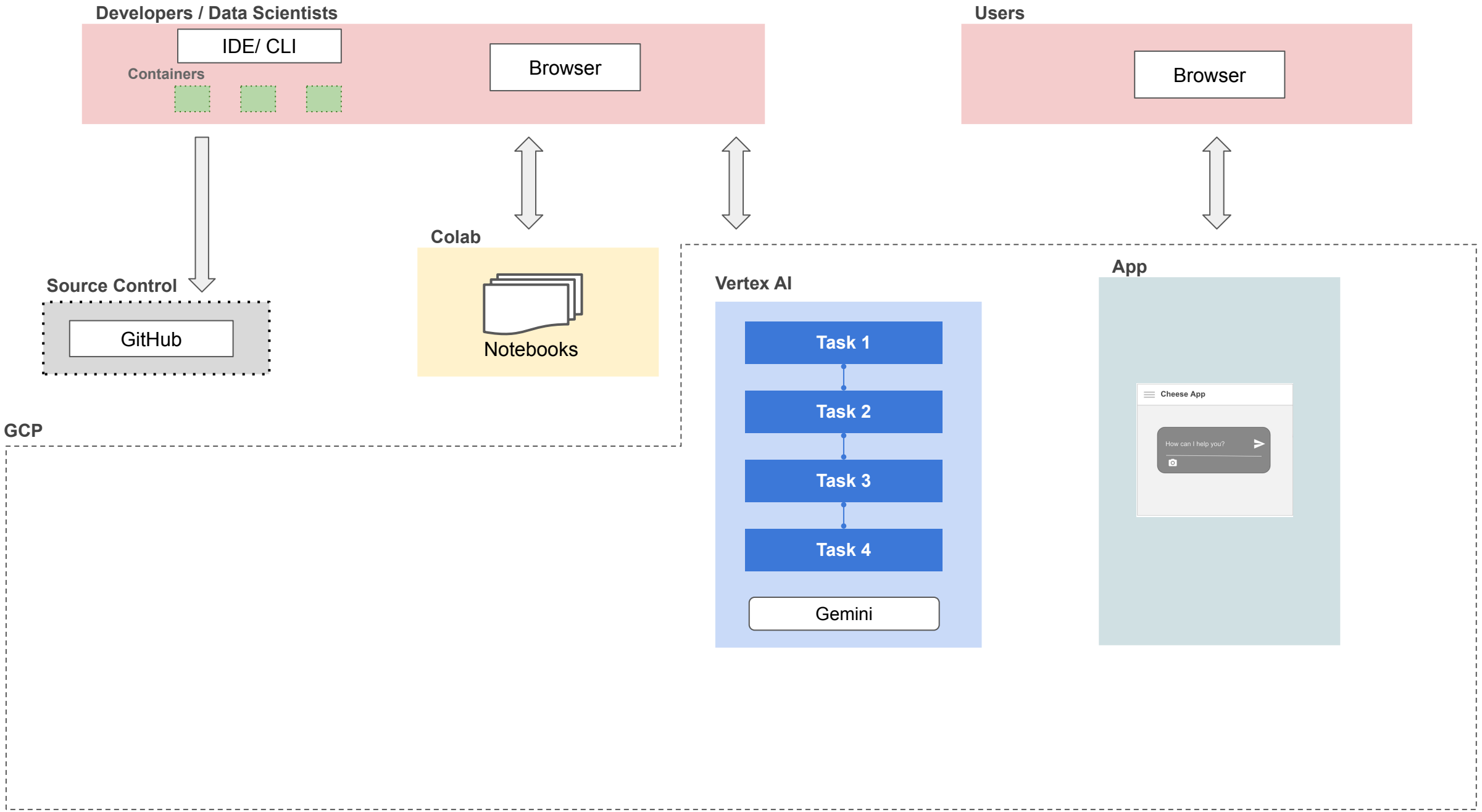
Users:

- Access the App using a browser
- Upload images and view prediction results
- Have a chat conversation

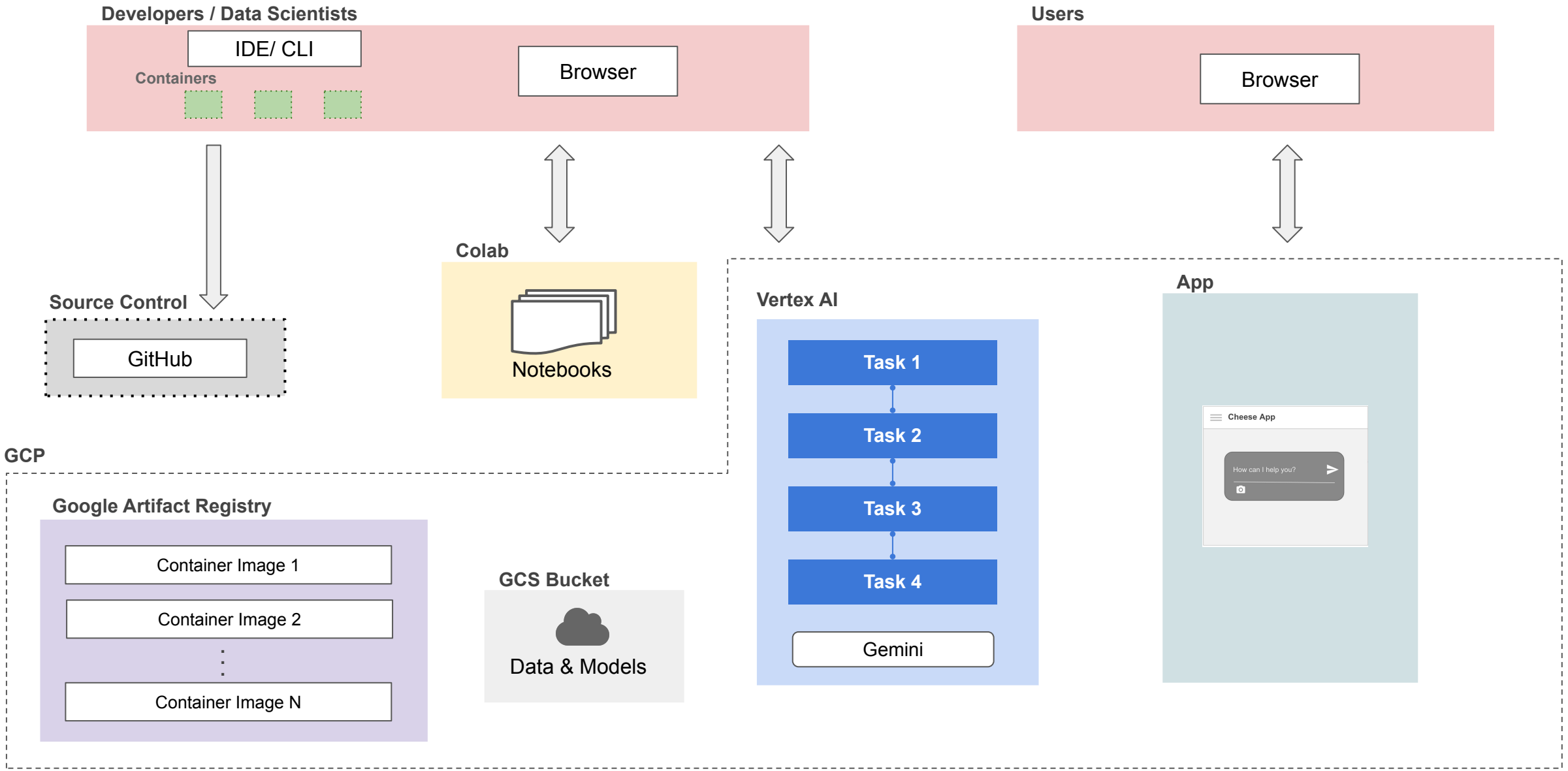
Building a Technical Architecture



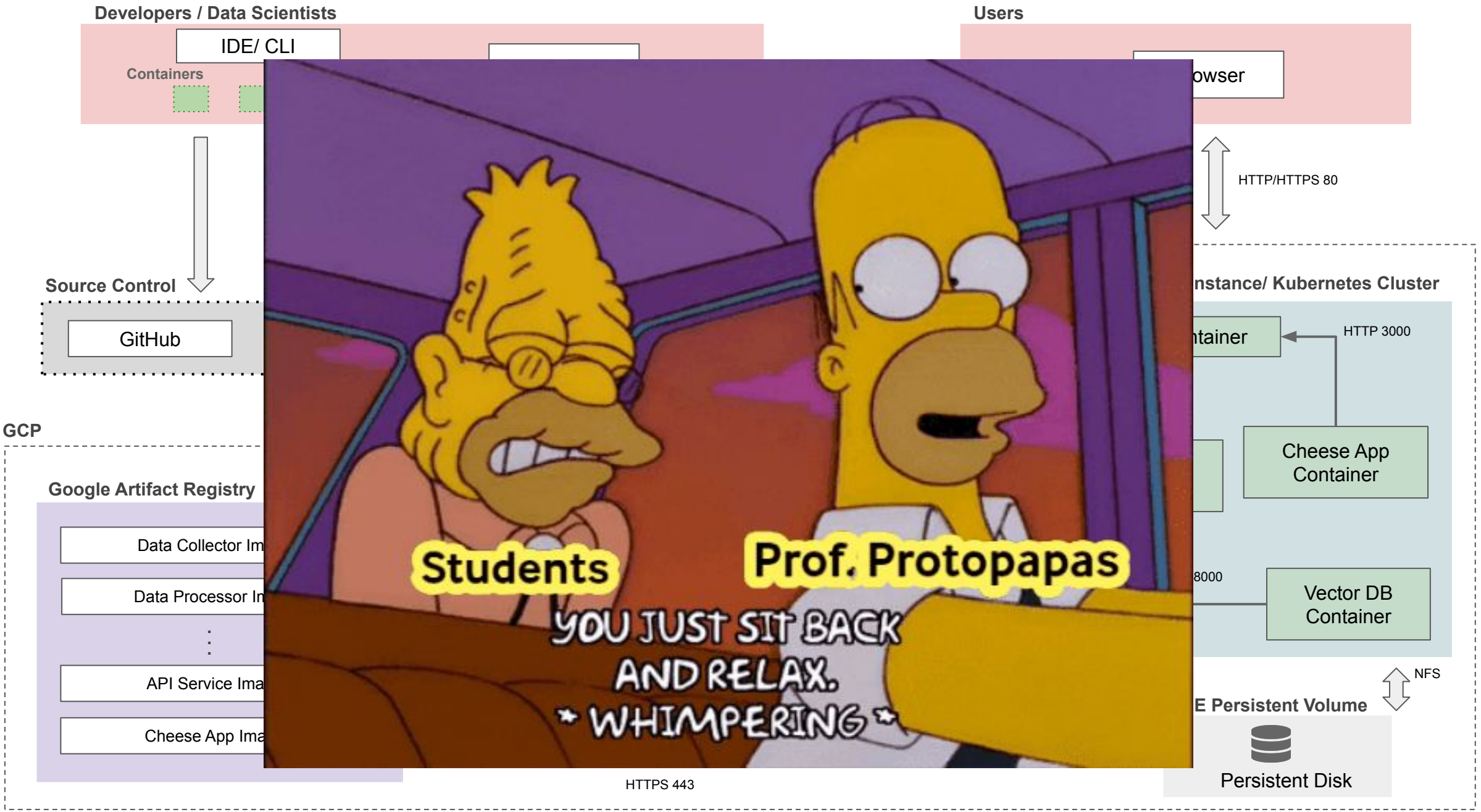
Building a Technical Architecture



Building a Technical Architecture



Technical Architecture



Technical Architecture Summary (GCP)

- **Source Control**
  - GitHub
- **Google Cloud Platform (GCP)**
  - GCP for deployment
- **Google Artifact Registry**
  - Host all the container images
- **GCS Buckets**
  - Storage buckets for models and model artifacts
  - Data store

- **Vertex AI**
  - Serverless ML Tasks
  - Vertex AI LLMs (Gemini, Claude)
- **GCE Persistent Volume**
  - Any files that need to be persisted when container images are updated
- **Compute Instance**
  - Hosting single instance of all containers
- **Kubernetes Cluster (GKE)**
  - Kubernetes cluster will be used to scale the app on GCP

Technical Architecture Summary (AWS)

- **Source Control**
  - GitHub
- **Amazon Web Service (AWS)**
  - AWS for deployment
- **Elastic Container Registry**
  - Host all the container images
- **S3 Buckets**
  - Storage buckets for models and model artifacts
  - Data store

- **Amazon SageMaker**
  - Serverless ML Tasks
  - Amazon Bedrock LLMs
- **Amazon Elastic Block Store**
  - Any files that need to be persisted when container images are updated
- **EC2 Instance**
  - Hosting single instance of all containers
- **Kubernetes Cluster (EKS)**
  - Kubernetes cluster will be used to scale the app on AWS

Technical Architecture Summary (Azure)

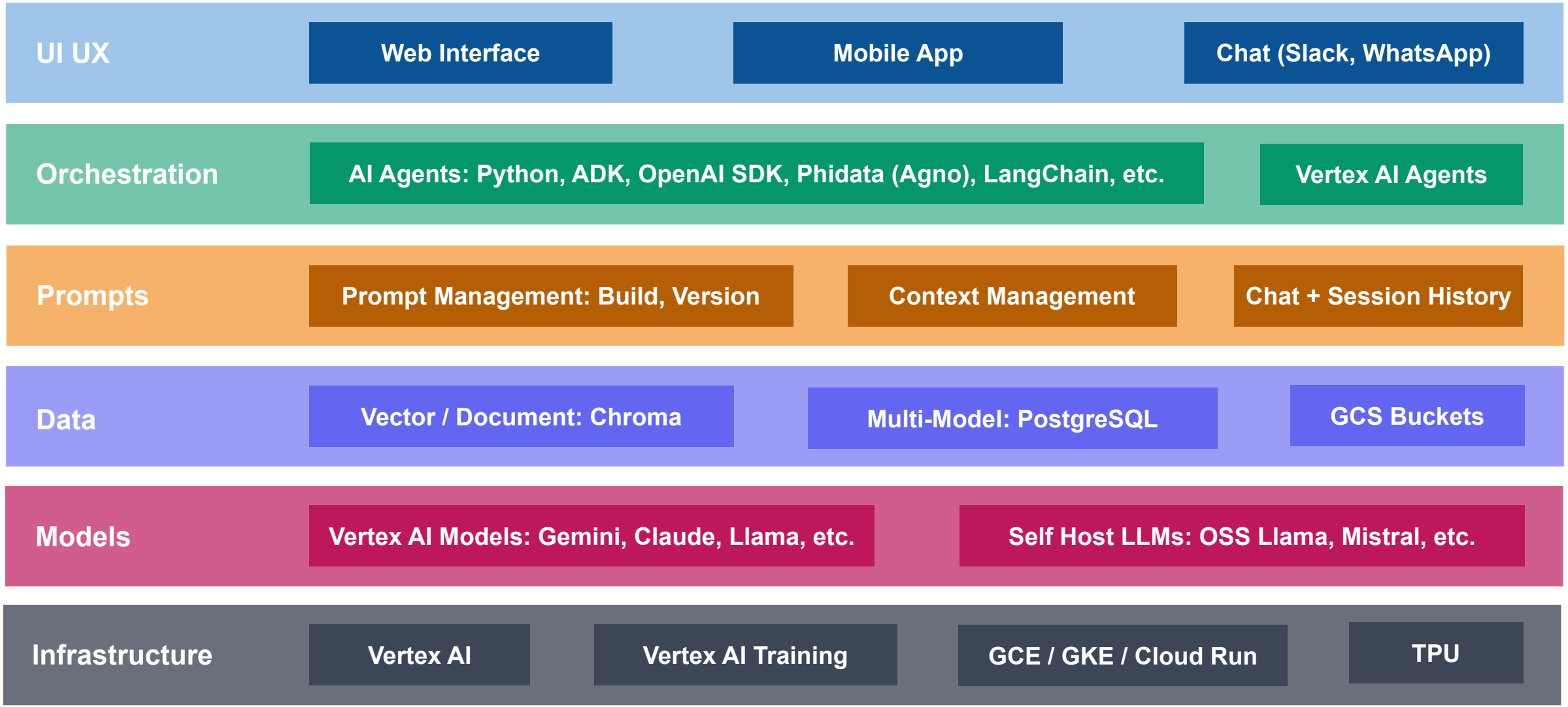
- **Source Control**
  - GitHub
- **Microsoft Azure**
  - Azure for deployment
- **Azure Container Registry (ACR)**
  - Host all the container images
- **Azure Blob Storage**
  - Storage containers for models and model artifacts
  - Data store

- **Azure Machine Learning**
  - Serverless ML Tasks
  - Azure OpenAI Service for LLMs
- **Azure Disk Storage**
  - Any files that need to be persisted when container images are updated
- **Azure Virtual Machines**
  - Hosting single instance of all containers
- **Kubernetes Cluster (AKS)**
  - Kubernetes cluster will be used to scale the app on Azure

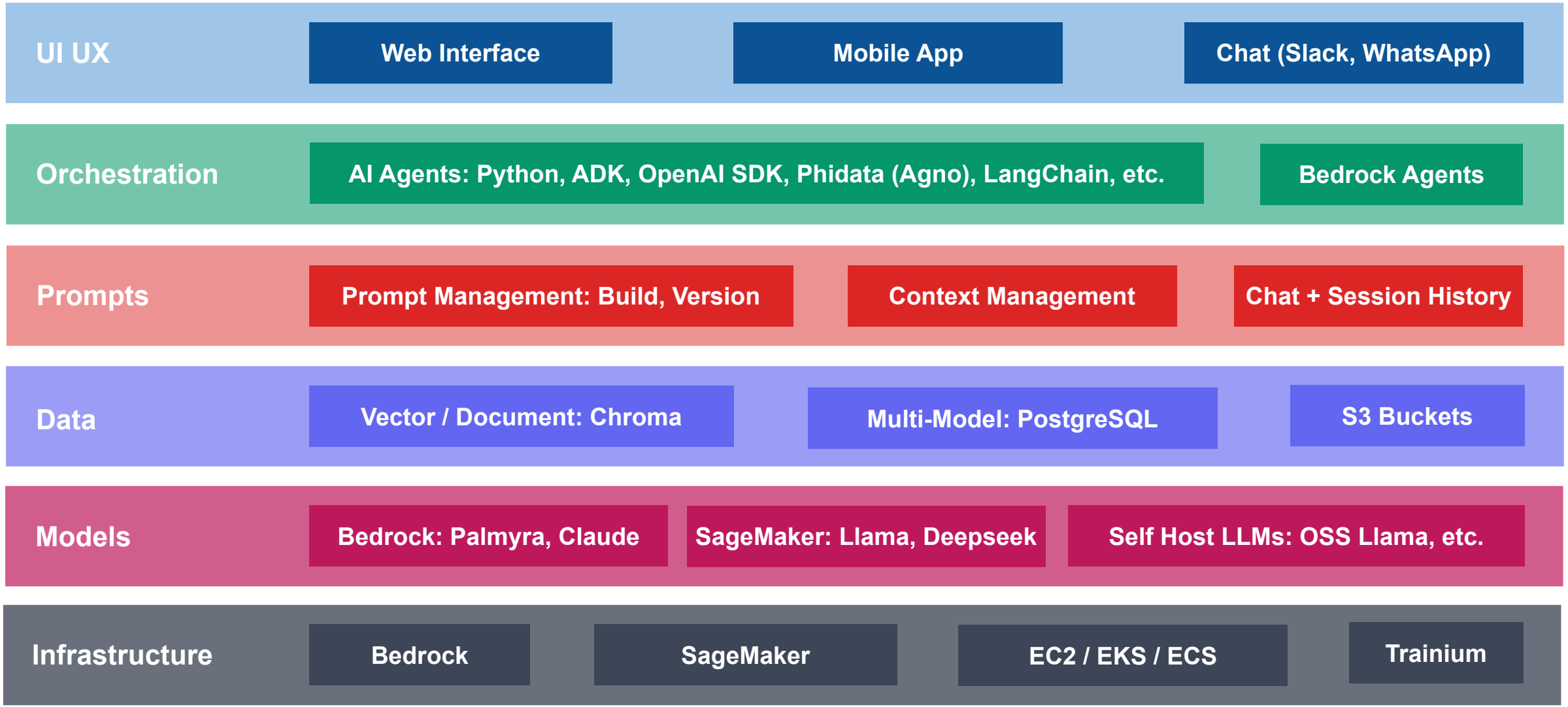
Technical Architecture Building Blocks (Comparison)

	Google Cloud Platform (GCP)	Amazon Web Services (AWS)	Microsoft Azure
Container Registry	Google Artifact Registry	Elastic Container Registry	Azure Container Registry
Object Storage	Google Cloud Storage	S3 Bucket	Azure Blob Storage
AI / ML Platform	Vertex AI	Amazon SageMaker	Azure Machine Learning
LLMs	Vertex AI LLMs	Bedrock LLMs	Azure OpenAI Service
Persistent Storage	GCE Persistent Volume	Elastic Block Store	Azure Disk Storage
Virtual Machines	Compute Instance	EC2 Instance	Azure Virtual Machine
Kubernetes Service	Google Kubernetes Engine	Elastic Kubernetes Service	Azure Kubernetes Service

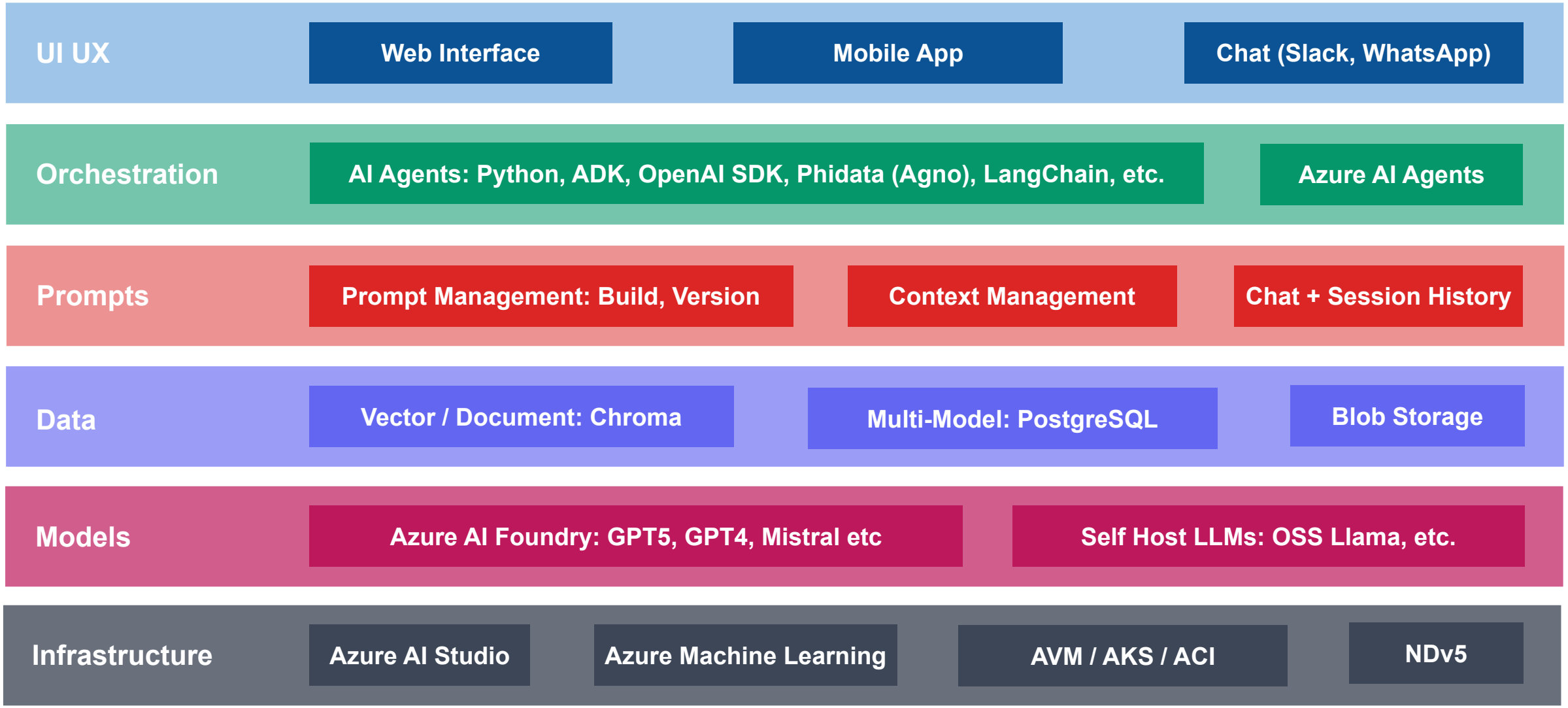
Agent Solutions Building Blocks (GCP)



Agent Solutions Building Blocks (AWS)



Agent Solutions Building Blocks (Azure)

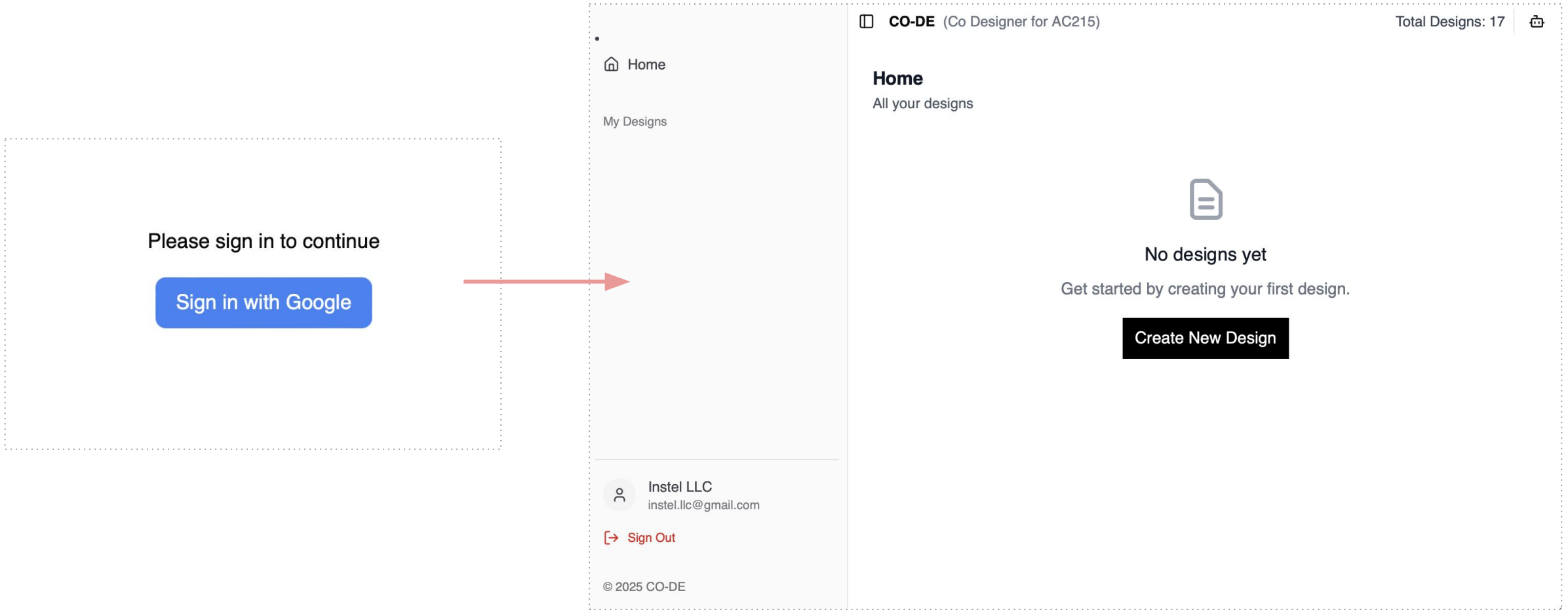


Tutorial: Designing an App

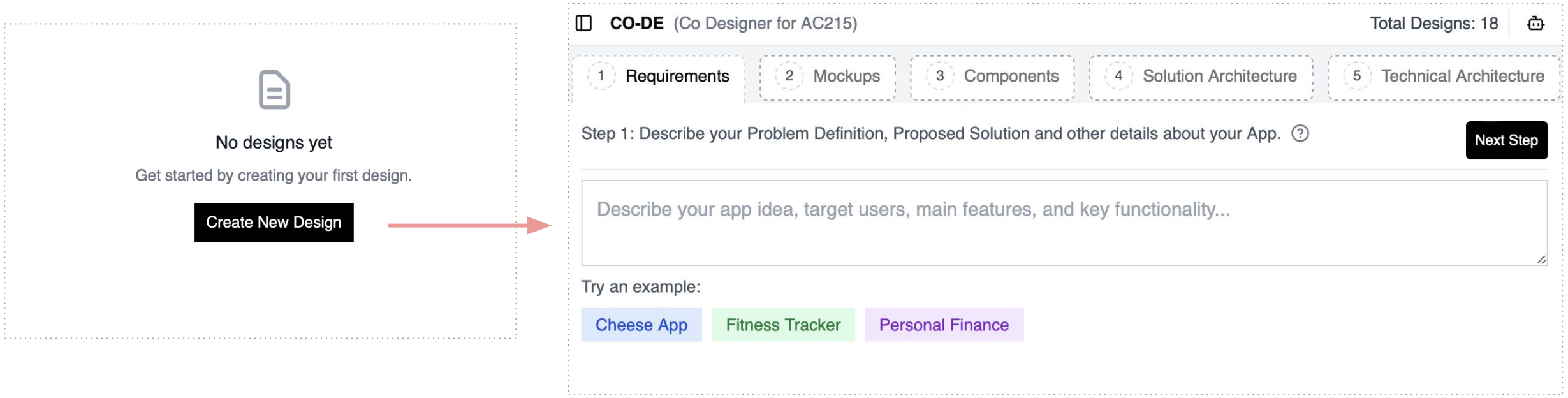
Steps to **Design an App**

- Go to the CO-DE App https://co-de.app/ .
- Sign in with your Google account
- Click “Create New Design”
- Copy your MS1 text into the Step 1: Requirements
- Click “Next Step”

Tutorial: Designing an App



Tutorial: Designing an App



Tutorial: Building Solution Architecture

Steps to build a **Solution Architecture**

- You will work with your project group
- Go to

  https://docs.google.com/presentation/d/1O1j-zcud3vTnD_dZpNPSddYpM8eoMRJ9Cds2wWzjEuU/edit?usp=sharing .
- Duplicate Slides 2 and 3 to the end of the slides.
- Put your group name in the slides.
- Identify Process, Execution, State for your project.
- Work on Solution Architecture slide for your project.

Outline

1. Recap
2. Motivation
3. App Design
4. Screenflow & Wireframes
5. Solution Architecture
6. Technical Architecture
7. **Setup & Code Organization**

Setup & Code Organization

- Think in **microservices** vs monolithic
- What determines how micro to get?
- Map **components** identified in your design to **microservices**
- Keep secrets outside of your git repo

Setup & Code Organization

```
/application-root-folder
   /src
        /api-service
        /data-collector
        /data-processor
        /model-training
        /model-deploy
        /ml-workflow
        /frontend-simple
        /frontend-react
        /vector-db
        /postgres-db
/secrets
   - data-service.json
```



Github root



Secrets (outside git folder)

THANK YOU